

Data Augmentation (resumo)

Data Augmentation se trata de uma técnica de aumento de dados, que objetiva melhorar a capacidade de generalização do modelo através de uma maior quantidade de dados disponíveis. Útil em pequenas bases de dados, em que a coleta de novas informações pode ser trabalhosa.

Importância do Data Augmentation em Deep Learning

Essa técnica se mostra relevante em deep learning considerando a propoção do desempenho percebida com o aumento da quantidade de dados disponíveis. Com a aplicação de data augmentation, melhora-se a capacidade de generalização do modelo, reduzindo overfitting, e aumenta a robustez, permitindo que se lide melhor com pequenas variações nas imagens, tais como rotação, deslocamento, ruído, iluminação, coloração e afins.

Exemplos de técnicas de aumento de dados;

No contexto de visão computacional, algumas técnicas envolvem recortes, redimensionamentos, espelhamentos, alteração de cores, brilho e contraste, remoção de áreas da imagem, alterações de perspectivas e rotações

Experimentos

1. Rotação aleatória

```
trans =  
transforms.Compose([transforms.RandomRotation(15), transforms.  
ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])
```

2. Nova estratégia de transformação:

```
trans_aumentado = transforms.Compose([  
    transforms.RandomAffine(  
        degrees = 0,  
        translate = (0.1, 0.1), # translação  
horizontal/vertical de até 10%  
        scale = (0.9, 1.1) # zoom entre 90 e 110%  
    ),  
    transforms.RandomPerspective(  
        distortion_scale = 0.2,  
        p = 0.5  
    ),  
    transforms.ToTensor(),  
    transforms.Normalize((0.1307,), (0.3081,))  
)
```

```

}))

# Train/Test Datasets
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans, download=True)
train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_aumentado, download=True)

```

Com a mudança na transformação foi observada uma dificuldade inicial de se alcançar uma boa acurácia, indicando a baixa previsibilidade do dataset. No entanto, o desempenho apresenta melhora com o passar das epochs.

3. Espelhamento horizontal no MNIST

Essa transformação não seria aplicável ao tipo de dado trabalhado nesse experimento, considerando que poderia haver uma distorção quanto à representação correta dos números. Nesse caso, a rotação dos dígitos pode afetar completamente seu significado, como no caso de 9 e 6, que se rotacionados igualam-se.

4. Comparação entre pipelines de treinamento

```

# Pequena transformação para tensores e normalizando o tamanho
trans = transforms.Compose([transforms.ToTensor(),
transforms.Normalize((0.1307,), (0.3081,))])

trans_aumentado = transforms.Compose([
    transforms.RandomAffine(

```

```

        degrees = 0,
        translate = (0.1, 0.1), # translação horizontal/vertical de
até 10%
        scale = (0.9, 1.1) # zoom entre 90 e 110%
    ),
    transforms.RandomPerspective(
        distortion_scale = 0.2,
        p = 0.5
    ),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

trans_avancado = transforms.Compose([
    transforms.TrivialAugmentWide(), # escolhe aleatoriamente uma
augmentation forte
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)),
    transforms.RandomErasing(
        p=0.25, # probabilidade de apagar parte da
imagem
        scale=(0.02, 0.2), # tamanho da região apagada
        ratio=(0.3, 3.3), # proporção da região
        value=0 # preenche com preto
    )
])

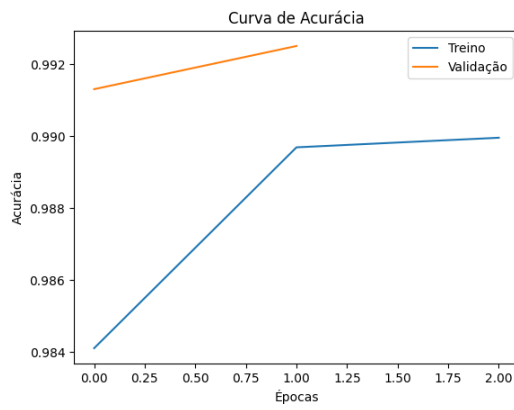
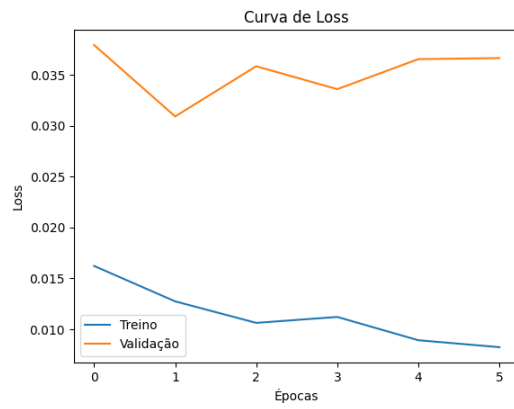
# Train/Test Datasets
train_dataset_basico = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans, download=True)

train_dataset_aumentado = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_aumentado, download=True)

train_dataset_avancado = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_avancado, download=True)

```

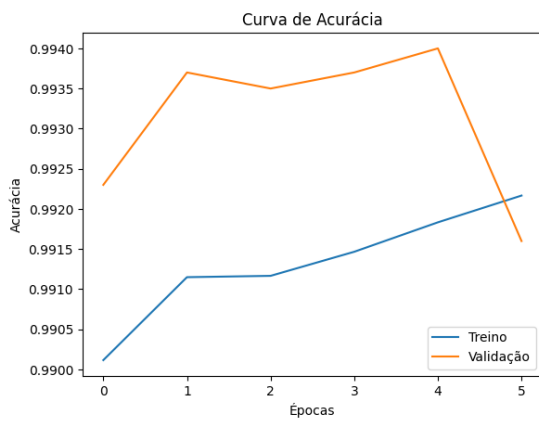
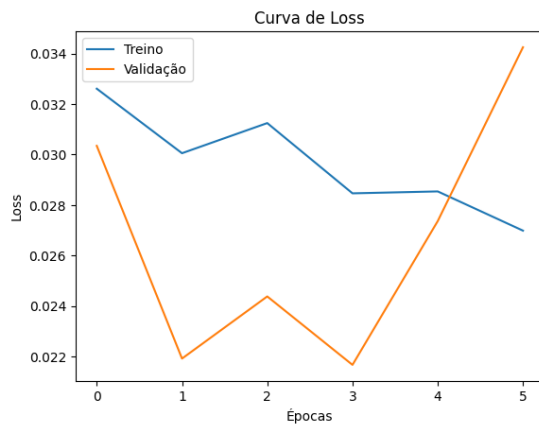
a. Modelo Básico



Acurácia final: 99.3

Tempo: 2 min

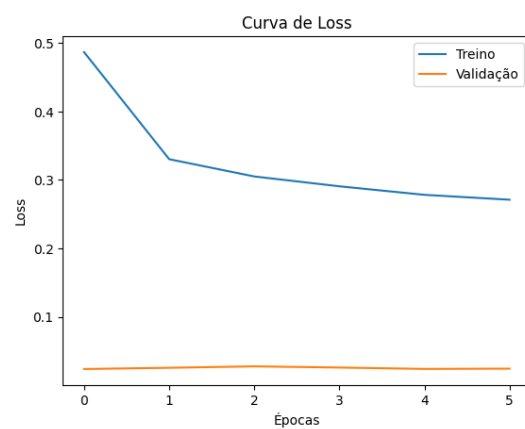
b. Modelo Aumentado

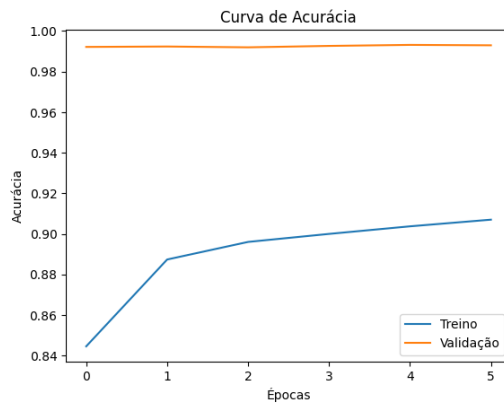


Acurácia final: 99.16

Tempo: 4 min

c. Modelo Avançado





Acurácia final: 99.29

Tempo: 4 min

5. Análise de resultados

- O aumento de dados melhorou a acurácia do modelo?
Não
- O treinamento ficou mais estável ou mais instável?
No modelo aumentado foi observada maior instabilidade
- Alguma estratégia prejudicou o desempenho?
Não, todas apresentaram bom desempenho final
- Qual pipeline apresentou melhor resultado?
O básico original, possivelmente por já ter uma quantidade considerável de dados para se trabalhar, não se fazendo necessária a data augmentation
- Qual estratégia você escolheria para esse problema e por quê?
Considerando os resultados finais de acurácia e a estabilidade apresentada, unida ao tempo de execução, optaria pelo modelo básico, que obteve melhor desempenho e um menor tempo de treinamento